

4. Formatos de datos en AJAX

Cuando hacemos una petición AJAX, el servidor puede devolver distintos **formatos de datos**. Los más comunes son:

1. **HTML** → Contenido ya formateado, listo para mostrar.
2. **JSON (JavaScript Object Notation)** → Datos estructurados que podemos procesar en JavaScript.
3. **XML (Extensible Markup Language)** → Formato más antiguo, jerárquico y estructurado.

4.1 HTML

- El servidor devuelve código HTML que podemos insertar directamente en la página.
- Es útil cuando queremos **actualizar partes de la página** sin reconstruir todo el DOM.

Ejemplo práctico: Cargar un fragmento HTML

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX con HTML</title>
</head>
<body>
  <h1>Noticias</h1>
  <button id="btnCargar">Cargar noticias</button>
  <div id="contenido"></div>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript (script.js)

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  let xhr = new XMLHttpRequest();
  xhr.open("GET", "noticias.html", true);
  xhr.onload = function() {
    if (xhr.status === 200) {
      document.getElementById("contenido").innerHTML =
xhr.responseText;
    } else {
      document.getElementById("contenido").innerText = "Error al
cargar noticias";
    }
  };
  xhr.send();
});
```

```
});
```

Archivo `noticias.html`

```
<ul>
  <li>Noticia 1: Estreno de película</li>
  <li>Noticia 2: Evento tecnológico</li>
  <li>Noticia 3: Campeonato de fútbol</li>
</ul>
```

Funcionamiento:

- Al hacer clic, el contenido HTML se carga directamente en el `div` sin recargar la página.

4.2 JSON

- Formato ligero y fácil de procesar en JavaScript.
- Muy usado para **APIs** y **aplicaciones modernas**.

Ejemplo práctico: Mostrar lista de usuarios desde JSON

Archivo `usuarios.json`

```
[
  {"nombre": "Ana", "edad": 25},
  {"nombre": "Luis", "edad": 30},
  {"nombre": "María", "edad": 28}
]
```

JavaScript (`script.js`)

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  fetch("usuarios.json")
    .then(response => response.json())
    .then(usuarios => {
      let lista = "<ul>";
      usuarios.forEach(u => {
        lista += `<li>${u.nombre} - ${u.edad} años</li>`;
      });
      lista += "</ul>";
      document.getElementById("contenido").innerHTML = lista;
    })
    .catch(error => console.error("Error:", error));
});
```

Ventaja:

- Fácil de transformar en objeto JavaScript.
- Permite manipular los datos antes de mostrarlos.

4.3 XML

- Formato más antiguo, jerárquico, menos usado hoy.
- Puede ser procesado usando `responseXML` de `XMLHttpRequest`.

Ejemplo práctico: Leer usuarios desde XML

Archivo `usuarios.xml`

```
<usuarios>
  <usuario>
    <nombre>Ana</nombre>
    <edad>25</edad>
  </usuario>
  <usuario>
    <nombre>Luis</nombre>
    <edad>30</edad>
  </usuario>
</usuarios>
```

JavaScript (`script.js`)

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  let xhr = new XMLHttpRequest();
  xhr.open("GET", "usuarios.xml", true);
  xhr.onload = function() {
    if (xhr.status === 200) {
      let xml = xhr.responseXML;
      let usuarios = xml.getElementsByTagName("usuario");
      let lista = "<ul>";
      for (let i = 0; i < usuarios.length; i++) {
        let nombre =
usuarios[i].getElementsByTagName("nombre")[0].textContent;
        let edad =
usuarios[i].getElementsByTagName("edad")[0].textContent;
        lista += `<li>${nombre} - ${edad} años</li>`;
      }
      lista += "</ul>";
      document.getElementById("contenido").innerHTML = lista;
    } else {
      document.getElementById("contenido").innerText = "Error al
cargar XML";
    }
  };
  xhr.send();
});
```

Observación:

- XML requiere más código para extraer los datos comparado con JSON.
- Hoy en día, **JSON es preferido** para aplicaciones web modernas.

4.4 Comparación de formatos

Formato	Fácil de procesar	Tamaño	Modernidad	Uso típico
HTML	Medio	Grande	Medio	Fragmentos de página
JSON	Muy fácil	Pequeño	Muy actual	APIs, datos estructurados
XML	Difícil	Grande	Antiguo	Documentos jerárquicos, legacy

4.5 Ejercicios propuestos

1. Modificar el ejemplo de HTML para cargar varias secciones (`header`, `footer`) desde archivos separados usando AJAX.
2. Crear un archivo JSON con 5 productos y mostrarlos en una tabla usando Fetch API.
3. Convertir el ejemplo de usuarios XML a JSON y mostrarlo usando Fetch API.